MS&E 228: Applied Causal Inference Powered by ML and AI

Lecture 9: Double Machine Learning in Practice

Vasilis Syrgkanis

Stanford University

Winter 2026

Readings: *Applied Causal Inference Powered by ML and AI*, §9.

Part I: Wrapping up Double ML

Key Theoretical Properties

Goals for Today

1. **Complete the DML algorithm** with cross-fitting and understand its asymptotic properties.
2. **Compare DML vs. Doubly Robust:** When to use which?
3. **Apply DML to panel data:** cluster-aware cross-fitting and cluster-robust standard errors.
4. **Estimate price elasticity of demand** using real Amazon data.

Where We Left Off

Last lecture, we laid the groundwork for Double Machine Learning:

- We introduced the **Partial Linear Model**: $Y = \theta D + f(X) + \epsilon$.
- We generalized the FWL theorem: we can residualize Y and D using **any ML method** to estimate θ.
- We showed that the estimating moment is **insensitive** to first-stage estimation errors, even for flexible ML models.
- We identified the risk of **overfitting bias**, which necessitates using separate data for nuisance model training and final estimation.

Today, we formalize the complete algorithm with cross-fitting and put it to work on real-world data.

The Double Machine Learning Algorithm

Algorithm (Double ML with K-fold Cross-Fitting)

1. Split data into K folds
2. For each fold k:
   - Train $\hat{h}^{(-k)}(X)$ on out-of-fold data to predict Y from X
   - Train $\hat{p}^{(-k)}(X)$ on out-of-fold data to predict D from X
   - For each sample i in fold k: $\check{Y}_i = Y_i - \hat{h}^{(-k)}(X_i)$, $\check{D}_i = D_i - \hat{p}^{(-k)}(X_i)$
3. Run simple OLS on all residuals (no intercept):

$$\hat{\theta} = \frac{\mathbb{E}_n[\check{Y}\check{D}]}{\mathbb{E}_n[\check{D}^2]}$$

Python Pseudocode for Double ML

```
from sklearn.model_selection import KFold

def double_ml(Y, D, X, n_folds=5):
    kf = KFold(n_splits=n_folds, shuffle=True)
    Y_res, D_res = np.zeros(len(Y)), np.zeros(len(Y))

    for train, test in kf.split(X):
        # Train on out-of-fold, predict for in-fold;
        # AutoML is placeholder for any ML method
        h_model = AutoML().fit(X[train], Y[train])
        Y_res[test] = Y[test] - h_model.predict(X[test])

        p_model = AutoML().fit(X[train], D[train])
        D_res[test] = D[test] - p_model.predict(X[test])

    # Final line-fit on surprises
    theta_hat = np.mean(Y_res * D_res) / np.mean(D_res**2)
    return theta_hat
```

Asymptotic Normality of Double ML

Theorem

Under regularity conditions, the cross-fitted DML estimator is asymptotically normal:

$$\sqrt{n}(\hat{\theta} - \theta_0) \xrightarrow{d} N\left(0, \frac{\mathbb{E}[(\tilde{Y} - \theta\tilde{D})^2\tilde{D}^2]}{\mathrm{Var}(\tilde{D})^2}\right)$$

Required conditions (the ML models need to be "good enough"):

1. Product rate: $\sqrt{n} \cdot \mathrm{RMSE}(\hat{h}) \cdot \mathrm{RMSE}(\hat{p}) \to 0$
2. Treatment model rate: $n^{1/4} \cdot \mathrm{RMSE}(\hat{p}) \to 0$
3. Consistency: $\mathrm{RMSE}(\hat{h}) \to 0$

We saw already how Random forests, Lasso, Neural Nets typically satisfy these conditions under small "effective dimension" assumptions.

The Overlap Analogue in Partial Linearity

Key Insight

The overlap condition becomes $\mathrm{Var}(\tilde{D}) > \epsilon$

- $\tilde{D} = D - \mathbb{E}[D|X]$ is the unpredictable part of treatment
- We need this to have positive variance **on average**
- Much weaker than pointwise overlap!

For binary treatments: $\mathrm{Var}(\tilde{D}) = \mathbb{E}[p(X)(1 - p(X))]$

- We need propensity bounded away from 0 and 1 **on average**
- Can extrapolate across covariate regions under partial linearity

Standard Errors for Double ML

Estimate the asymptotic variance using empirical analogues:

$$\hat{\sigma}^2 = \frac{\mathbb{E}_n[(\check{Y} - \hat{\theta}\check{D})^2 \check{D}^2]}{\mathbb{E}_n[\check{D}^2]^2}$$

Standard error: s.e. $= \hat{\sigma}/\sqrt{n}$

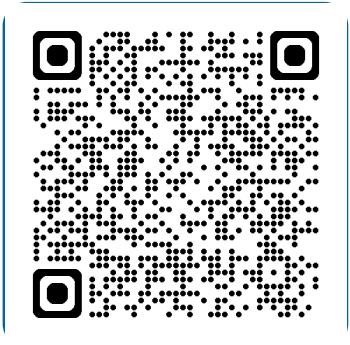
```
def double_ml_with_se(Y, D, X, n_folds=5):
    theta_hat, Y_res, D_res = double_ml(Y, D, X, n_folds)

    # Compute standard error
    epsilon = Y_res - theta_hat * D_res
    sigma_sq = np.mean(epsilon**2 * D_res**2) / np.mean(D_res**2)**2
    se = np.sqrt(sigma_sq / len(Y))

    return theta_hat, se
```

Live Coding: Double ML for Price Elasticities (Simple Version)

Exercise

Goal: Implement DML from scratch with cross-fitting.

1. Implement cross-fitting loop
2. Use FLAML AutoML to find best models
3. Compute residuals for each fold
4. Run final line-fit on residuals
5. Compute standard errors
6. Compare with true elasticity!



Open notebook and follow along!

Key insight: DML recovers the true effect even when $f(X)$ is nonlinear!

Part II: DML vs. Doubly Robust

A Tale of Two Insensitive Estimators

A Tale of Two Insensitive Estimators

Both the Doubly Robust (DR) estimator and the Double Machine Learning (DML) estimator are built on the powerful idea of using an **insensitive** or **orthogonal** moment equation. This property allows us to use complex machine learning models for nuisance components while still achieving valid statistical inference for our causal parameter of interest. However, they are designed for different settings and target slightly different parameters.

Understanding their respective strengths and weaknesses is key to choosing the right tool for your causal inference problem.

In-Class Discussion

Group Discussion

Turn to your neighbors and discuss for the next 10 minutes. What are the pros and cons between the DR vs DML estimator?

In-Class Discussion

Case Studies (Respond in Poll Everywhere `https://pollev.com/vsyrgkanis`)

1. You are an economist at Amazon, and your manager wants to know the effect of a small price change on sales. Your treatment variable is **price (continuous)**. Which estimator is more natural to use, and why?

2. You are analyzing a get-out-the-vote campaign. For a certain subgroup of your data (e.g., young voters in a specific state), almost everyone received the treatment (a reminder to vote). You are worried about **poor overlap**. Which estimator might provide a more stable estimate?

3. You suspect that the effect of a new drug is not constant but **varies significantly** based on patient characteristics (age, gender, etc.). What does the DML estimator give you in this case of heterogeneous effects? How does it differ from the ATE estimated by DR?

Key Takeaways from the Comparison

Which Estimator Should I Use?

- **Use the Doubly Robust (DR) estimator when:**
  - Your treatment is **binary**.
  - Primary goal is to estimate population **Average Treatment Effect (ATE)**.
  - You have good overlap across the entire covariate space.

- **Use the Double Machine Learning (DML) estimator when:**
  - Your treatment is **continuous** (e.g., price, dosage, time spent).
  - The partial linearity assumption seems plausible for your application.
  - You are concerned with **robustness to poor overlap** (extreme propensities).

Common Ground: Both estimators are powerful, modern techniques that leverage machine learning for nuisance components and rely on cross-fitting and an insensitivity property to deliver valid causal inference.

Part III: In-Class Activity

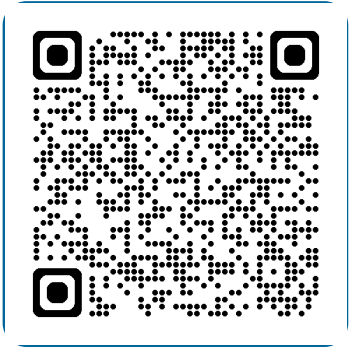
Adventures in Demand Analysis Using AI

In-Class Activity: Estimating Price Elasticity in Panel Data (Advanced)

Live Coding Exercise

Goal: Apply DML to a real-world dataset to estimate the price elasticity of demand for toy cars on Amazon.

1. Confront endogeneity and the need for clustered standard errors.
2. Implement cluster-aware cross-fitting to prevent data leakage.
3. Manually build the DML estimator step-by-step.
4. Compare our manual estimate to the `doubleml` library implementation.



Open notebook and follow along!

The Data and The Goal

Dataset: Amazon toy car products tracked over approximately one year.

- **Panel Data:** 3,800 unique products (ASINs), with multiple observations over time, for a total of 38,000 observations.

Key Variables:

- **Outcome (Y):** `Q_t` = log(1/Sales Rank), a proxy for log quantity sold.
- **Treatment (D):** `P_bb_t` = log(Buybox Price).
- **Controls (X):** A rich set including lagged quantity and price, time dummies (for seasonality), and high-dimensional text embeddings from product descriptions.

Goal

Our objective is to estimate the **price elasticity of demand**, which is the coefficient on the log-price treatment variable.

Challenge 1: Endogeneity (Confoundedness) of Price

Problem: Unobserved, persistent factors (brand quality) affect both price and demand.

Solution: Control for many variables that capture the persistent quality of a product.

```
# Naive OLS (with clustered SEs)
ols_naive = sm.OLS(Y, D).fit()
# Result -> Estimate: -0.11 (Implausibly small elasticity)

# OLS with lagged controls (with clustered SEs)
ols_lags = sm.OLS(Y, [D, X_lags]).fit()
# Result -> Estimate: -0.69 (More plausible, importance of controls)
```

Discussion Point

Why does adding lagged quantity and price as controls change the elasticity estimate so dramatically from -0.11 to -0.69?

Challenge 2: Endogeneity and Clustered Errors

Problem: Observations for the same product, across time, are not independent.

Solution: We must use **cluster-robust standard errors** to avoid false precision.

```
# Naive OLS (with clustered SEs)
ols_naive = sm.OLS(Y, D).fit(cov_type='cluster',
                             cov_kwds={'groups': df['ASIN']})
# Result -> Estimate: -0.11 (Implausibly small elasticity)

# OLS with lagged controls (with clustered SEs)
ols_lags = sm.OLS(Y, [D, X_lags]).fit(cov_type='cluster',
                                      cov_kwds={'groups': df['ASIN']})
# Result -> Estimate: -0.69 (More plausible,importance of controls)
```

Discussion Point

What would be the problem if we don't use clustered standard errors?

Challenge 3: Data Leakage in Cross-Fitting

Problem: Standard `KFold` cross-validation will randomly split individual observations. This means data from the *same product* can appear in both training and testing sets.

Why this is bad: The model can "cheat" by learning the specific, time-invariant features of a product in the training set and using that knowledge to predict its outcome in the test set. This leads to **overfitting and biased estimates**.

Solution: Use **Cluster-Aware Cross-Fitting** with `GroupKFold`, which ensures that all observations from a single product (group) are kept in the same fold.

```
from sklearn.model_selection import GroupKFold

# Ensures all observations for a given product (group)
# are in the same fold
gkf = GroupKFold(n_splits=5, shuffle=True, random_state=42)
for train, test in gkf.split(X, Y, groups=df['ASIN']):
    # ... proceed with training on train and predicting on test
```

Putting It All Together: DML with Clustering

Here is the manual implementation combining both key adjustments for panel data:

```
# 1. Loop with GroupKFold to ensure cluster-aware splitting
for train, test in GroupKFold(n_splits=5).split(X, Y, groups):
    # 2. Train nuisance models on train data
    model_y.fit(X[train], Y[train])
    model_d.fit(X[train], D[train])

    # 3. Predict on test data and get residuals
    Y_resid[test] = Y[test] - model_y.predict(X[test])
    D_resid[test] = D[test] - model_d.predict(X[test])

# 4. Final OLS on residuals with clustered standard errors
dml_est = sm.OLS(Y_resid, D_resid).fit(
    cov_type='cluster', cov_kwds={'groups': groups})
```

Using the doubleml Library

While manual implementation is instructive, professional libraries make DML much easier and safer. The doubleml package handles all the details of cluster-aware cross-fitting and estimation automatically.

```
import doubleml as dml

# 1. Create DoubleMLData object, specifying the cluster variable
dml_data = dml.DoubleMLData(df, y_col='Q_t', d_cols='P_bb_t',
                            x_cols=control_vars, cluster_cols='ASIN')
# 2. Initialize and fit DML model (PLR = Partially Linear Regression)
dml_plr = dml.DoubleMLPLR(dml_data, ml_l=LGBM(), ml_m=LGBM())
dml_plr.fit()

# 3. Get a full summary of the results
print(dml_plr.summary)
```

Key Takeaways from the In-Class Activity

Summary of Findings

1. **Endogeneity is a Major Threat:** Naive OLS is severely biased due to confounding. Controlling for confounders is critical for causal inference.

2. **Clustering is Essential for Panel Data:** For panel data, you must use cluster-robust standard errors and cluster-aware cross-fitting (`GroupKFold`).

3. **DML is a Practical and Powerful Tool:** The DML algorithm, especially when implemented via robust libraries, provides a reliable method for estimating causal effects in complex, real-world settings with high-dimensional confounders.

4. **Found a Plausible Estimate:** The DML price elasticity of approximately -0.73 is economically plausible and statistically significant, demonstrating the power of the method to recover a meaningful causal estimate from messy observational data.

Four Key Takeaways from Today's Lecture

1. **The Full DML Algorithm:** DML combines cross-fitting with residualization to provide asymptotically normal estimates of causal parameters, even when using flexible ML models for nuisance components.
2. **DML vs. DR:** DML is the natural choice for continuous treatments and is robust to overlap violations, while DR directly targets the population ATE for binary treatments.
3. **DML for Panel Data:** This requires two key adjustments: cluster-aware cross-fitting (`GroupKFold`) to prevent data leakage and cluster-robust standard errors to account for correlated observations.
4. **DML in Action:** We successfully applied DML to estimate a plausible price elasticity of demand from real-world Amazon data, overcoming the severe bias that plagued simpler methods.

References

- Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., & Robins, J. (2018). Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1), C1-C68.
- Syrgkanis, V., & Zetty, T. (2024). Adventures in Demand Analysis Using AI. *Working Paper*.
- EconML: A Microsoft library for causal machine learning. `https://econml.azurewebsites.net/`
- DoubleML: An open-source Python library for Double Machine Learning. `https://docs.doubleml.org/`

Appendix

Feature	Doubly Robust (DR)	Double ML (DML)
Treatment Type	Primarily for **binary** treatments.	Handles **continuous** or binary treatments naturally.
Target Estimand	The population Average Treatment Effect (ATE): $\mathbb{E}[Y(1) - Y(0)]$.	Coefficient θ in a PLR Model: $Y = \theta D + f(X) + \epsilon$. Without PLR, it estimates weighted average effect (e.g. overlap weighted ATE, weighted average derivative)
Key Assumption	Unconfoundedness: $Y(d) \perp D \mid X$.	Partial Linearity and Unconfoundedness.
Overlap Requirement	Requires **pointwise** overlap: $0 < P(D = 1\|X) < 1$ for all X.	Requires only **average** overlap: $\mathrm{Var}(D - \mathbb{E}[D\|X]) > 0$.
Key Property	**Double Robustness**: Consistent if either the outcome or propensity model is correct.	**Neyman Orthogonality (Insensitivity)**: First-stage errors don't affect final estimate's distribution.
Variance	Can have high variance in regions where the propensity score $p(X)$ is close to 0 or 1.	Naturally down-weights low-overlap regions, often leading to lower variance.

Part VIII: Beyond Partial Linearity

Flexible Dose-Response Curves, Heterogeneous Effects

Motivation: Nonlinear Dose-Response

Partial linearity assumes $\mathbb{E}[Y(d)|X] = \theta d + f(X)$—a linear dose-response.

But what if the dose-response is nonlinear?

- Drug effects may saturate at high doses
- Price elasticity may vary with price level
- Returns to investment may be diminishing

Generalized Partial Linear Model

$$Y = \theta' \phi(D, X) + f(X) + \epsilon, \quad \mathbb{E}[\epsilon | D, X] = 0$$

where $\phi(D, X)$ is a vector of basis functions (e.g., polynomials, splines). Under conditional exogeneity, this is equivalent to the assumption:

$$\mathbb{E}[Y(d) - Y(0) \mid X = x] = \theta' \phi(d, x)$$

Examples of Basis Functions

Polynomial basis: $\phi(D, X) = (D, D^2, D^3, \ldots, D^p)'$

$$\mathbb{E}[Y(d)|X] = \theta_1 d + \theta_2 d^2 + \cdots + \theta_p d^p + f(X)$$

Spline basis: $\phi(D, X) = (D, (D - k_1)_+, (D - k_2)_+, \ldots)'$

- Allows flexible, piecewise-linear dose-response
- $k_1, k_2, \ldots$ are knot locations

Interaction basis: $\phi(D, X) = (D, D \cdot X_1, D \cdot X_2, \ldots)'$

- Allows treatment effect to vary with covariates
- Relaxes constant effect assumption

DML for Generalized PLM

Algorithm (Generalized DML)

1. Split data into K folds
2. For each fold k:
   - Train $\hat{h}^{(-k)}(X)$ to predict Y from X
   - Train $\hat{q}^{(-k)}(X)$ to predict $\phi(D, X)$ from X (vector-valued)
   - Compute residuals: $\check{Y}_i = Y_i - \hat{h}^{(-k)}(X_i)$
   - Compute residuals: $\check{\phi}_i = \phi(D_i, X_i) - \hat{q}^{(-k)}(X_i)$
3. Run multivariate OLS on residuals:

$$\hat{\theta} = \left(\mathbb{E}_n[\check{\phi}\check{\phi}']\right)^{-1} \mathbb{E}_n[\check{\phi}\check{Y}]$$

Same idea: residualize the basis functions, then regress!

Python Pseudocode: Generalized DML

```
def generalized_dml(Y, D, X, phi_func, n_folds=5):
    """phi_func(D, X) returns basis matrix of shape (n, p)"""
    Phi = phi_func(D, X)  # (n, p) basis matrix
    n, p = len(Y), Phi.shape[1]
    Phi_res, Y_res = np.zeros((n, p)), np.zeros(n)

    for train, test in KFold(n_splits=n_folds, shuffle=True).split(X):
        # Outcome model
        h_model = AutoML().fit(X[train], Y[train])
        Y_res[test] = Y[test] - h_model.predict(X[test])
        # Basis residuals (fit each component)
        for j in range(p):
            q_model = AutoML().fit(X[train], Phi[train, j])
            Phi_res[test, j] = Phi[test, j] - q_model.predict(X[test])

    # Multivariate OLS on residuals
    theta_hat = np.linalg.solve(Phi_res.T @ Phi_res, Phi_res.T @ Y_res)
    return theta_hat, Y_res, Phi_res
```

DML for Generalized PLM with Separable Features

When $\phi(d, x) = \psi(x) \cdot d$, then residualization is easy:

$$\mathbb{E}[\phi(D, X) \mid X] = \psi(x)\mathbb{E}[D \mid X].$$

Algorithm (Generalized DML)

1. Split data into K folds
2. For each fold k:
   - Train $\hat{h}^{(-k)}(X)$ to predict Y from X
   - Train $\hat{p}^{(-k)}(X)$ to predict D from X
   - Compute residuals: $\check{Y}_i = Y_i - \hat{h}^{(-k)}(X_i)$
   - Compute residuals: $\check{\phi}_i = \psi(X_i) \cdot (D_i - \hat{p}^{(-k)}(X_i))$
3. Run multivariate OLS on residuals:

$$\hat{\theta} = \left(\mathbb{E}_n[\check{\phi}\check{\phi}']\right)^{-1} \mathbb{E}_n[\check{\phi}\check{Y}]$$

Same idea: residualize the basis functions, then regress!

Python Pseudocode: Generalized DML

```
def generalized_dml(Y, D, X, psi_func, n_folds=5):
    """psi_func(X) returns basis matrix of shape (n, p)"""
    Psi = psi_func(X)  # (n, p) basis matrix
    n, p = len(Y), Phi.shape[1]
    D_res, Y_res = np.zeros(n), np.zeros(n)

    for train, test in KFold(n_splits=n_folds, shuffle=True).split(X):
        # Outcome model
        h_model = AutoML().fit(X[train], Y[train])
        Y_res[test] = Y[test] - h_model.predict(X[test])
        # Treatment model
        p_model = AutoML().fit(X[train], D[train])
        D_res[test] = D[test] - q_model.predict(X[test])

    Phi_res = Psi * D_res.reshape(-1, 1)

    # Multivariate OLS on interacted residuals
    theta_hat = np.linalg.solve(Phi_res.T @ Phi_res, Phi_res.T @ Y_res)
    return theta_hat, Y_res, Phi_res
```

Asymptotic Distribution

Under regularity conditions, $\hat{\theta}$ is asymptotically normal:

$$\sqrt{n}(\hat{\theta} - \theta_0) \xrightarrow{d} N(0, \Sigma)$$

where Σ is given by the sandwich formula:

$$\Sigma = \left(\mathbb{E}[\tilde{\phi}\tilde{\phi}']\right)^{-1} \mathbb{E}[\tilde{\phi}\tilde{\phi}'\epsilon^2] \left(\mathbb{E}[\tilde{\phi}\tilde{\phi}']\right)^{-1}$$

Estimated covariance matrix: these is the HC0 robust covariance matrix that statsmodels packages calculate

$$\hat{\Sigma} = \left(\mathbb{E}_n[\check{\phi}\check{\phi}']\right)^{-1} \mathbb{E}_n[\check{\phi}\check{\phi}'\hat{\epsilon}^2] \left(\mathbb{E}_n[\check{\phi}\check{\phi}']\right)^{-1}$$

where $\hat{\epsilon}_i = \check{Y}_i - \hat{\theta}'\check{\phi}_i$.

Python Pseudocode: Covariance Estimation

```
def generalized_dml_with_se(Y, D, X, psi_func, n_folds=5):
    """Computes DML estimator for theta and its covariance matrix."""
    theta_hat, Y_res, Phi_res = generalized_dml(Y, D, X, psi_func, n_folds)

    # Compute residuals
    epsilon_hat = Y_res - Phi_res @ theta_hat

    # Compute HC0 robust covariance matrix
    # Formula: (X'X)^-1 (X' diag(e^2) X) (X'X)^-1
    phi_t_phi_inv = np.linalg.inv(Phi_res.T @ Phi_res)
    meat = Phi_res.T @ np.diag(epsilon_hat**2) @ Phi_res
    cov_matrix = (phi_t_phi_inv @ meat @ phi_t_phi_inv)

    # Standard errors from covariance matrix
    se = np.sqrt(np.diag(cov_matrix) / len(Y))

    return theta_hat, se, cov_matrix
```

Inference on Conditional Dose-Response Curve / Conditional ATE

Goal: Construct CI for

$$\tau(d, x) = \mathbb{E}[Y(d) - Y(0) \mid X = x]$$

at specific dose d and covariate value x.

Under the Generalized Partially Linear model:

$$\tau(d, x) = \theta' \underbrace{(\phi(d, x) - \phi(0, x))}_{\delta_{d,x}}$$

Point Estimate

$$\widehat{\tau}(d, x) = \hat{\theta}' \delta_{d,x}$$

Delta Method for Conditional Dose-Response / CATE CI

Since, $\sqrt{n}(\hat{\theta} - \theta_0) \Rightarrow N(0, \Sigma)$, we also have that:

$$\sqrt{n}(\hat{\tau}(d, x) - \tau(d, x)) = \sqrt{n}\delta_d'(\hat{\theta} - \theta_0) \Rightarrow N(0, \delta_{d,x}' \Sigma \delta_{d,x})$$

95% CI for Conditional Dose-Response / CATE at d

$$\hat{\theta}' \delta_{d,x} \pm 1.96 \cdot \sqrt{\frac{\delta_{d,x}' \hat{\Sigma} \delta_{d,x}}{n}}$$

Inference on Average Dose-Response Curve / ATE

Goal: Construct CI for $\tau(d) = \mathbb{E}[Y(d) - Y(0)]$ at a specific dose d.

Under the model: $\tau(d) = \theta' \underbrace{(\mathbb{E}[\phi(d, X)] - \mathbb{E}[\phi(0, X)])}_{\delta_d}$

Point Estimate

$$\widehat{\tau}(d) = \hat{\theta}' \bar{\delta}_d$$

where $\bar{\delta}_d = \mathbb{E}_n[\phi(d, X_i) - \phi(0, X_i)]$ estimates δ_d.

We estimate both θ and the population average of ϕ—both contribute uncertainty!

Delta Method for Average Dose-Response / ATE CI

Two sources of variance: We estimate both θ and $\delta_d = \mathbb{E}[\phi(d, X) - \phi(0, X)]$.

The full variance formula:

$$\mathrm{Var}(\hat{\tau}(d)) \approx \underbrace{\bar{\delta}_d' \hat{\Sigma} \bar{\delta}_d / n}_{\text{from estimating } \theta} + \underbrace{\mathrm{Var}_n(\hat{\theta}' \phi(d, X_i))/n}_{\text{from estimating } \mathbb{E}[\phi]}$$

95% CI for Average Dose-Response / ATE at d

$$\hat{\theta}' \bar{\delta}_d \pm 1.96 \cdot \sqrt{\frac{\bar{\delta}_d' \hat{\Sigma} \bar{\delta}_d + \mathrm{Var}_n(\hat{\theta}' \phi(d, X_i))}{n}}$$

Don't forget the second term—it captures heterogeneity in $\phi(d, X)$ across the population!

Python Example: Polynomial Dose-Response

```python
def poly_basis(D, degree=3):
    return np.column_stack([D**k for k in range(1, degree+1)])

def poly_basis_deriv(D, degree=3):
    return np.column_stack([k * D**(k-1) for k in range(1, degree+1)])

# Fit generalized DML
theta, se, cov = generalized_dml_with_se(Y, D, X,
                    lambda d, x: poly_basis(d, 3))

# Dose-response at d=2: include BOTH variance terms
d_eval = 2.0
delta_d = np.mean(poly_basis(d_eval, 3) - poly_basis(0, 3), axis=0)
dr_est = theta @ delta_d
var_theta_part = delta_d @ cov @ delta_d  # from theta
var_phi_part = np.var(poly_basis(d_eval, 3) @ theta)  # from E[phi]
dr_se = np.sqrt((var_theta_part + var_phi_part) / n)
print(f"E[Y({d_eval})] - E[Y(0)] = {dr_est:.3f} +/- {1.96*dr_se:.3f}")
```

Inference on Conditional Average Marginal Effect

Under the Generalized Partially Linear model:

$$\text{ME}(d, x) = \theta' \underbrace{\frac{\partial \phi(d, x)}{\partial d}}_{\psi_{d,x}}$$

95% CI for Conditional Average Marginal Effect at d, x

$$\hat{\theta}' \psi_{d,x} \pm 1.96 \cdot \sqrt{\frac{\psi'_{d,x} \hat{\Sigma} \psi_{d,x}}{n}}$$

Inference on Average Marginal Effect

The AME (averaged over observed D, X):

$$\text{AME} = \theta' \underbrace{\mathbb{E}\left[\frac{\partial \phi(D, X)}{\partial D}\right]}_{\psi}$$

95% CI for Average Marginal Effect

$$\hat{\theta}'\bar{\psi} \pm 1.96 \cdot \sqrt{\frac{\bar{\psi}'\hat{\Sigma}\bar{\psi} + \text{Var}_n\left(\hat{\theta}'\frac{\partial \phi(D_i, X_i)}{\partial D}\right)}{n}}$$

where $\bar{\psi} = \mathbb{E}_n\left[\frac{\partial \phi(D,X)}{\partial D}\right]$ estimates ψ.